

1. 我首先去读vllm的论文以及attention is all you need, 但是全英的, 就在b站和抖音找相关的up主, 看他们的讲解, 然后让codex把原文找出来, 在帮我从背景, 方法, 创新点, 局限性等角度去分析这篇论文, 看下是否有可以改进的地方。对于SGLang, 这个代码实现的部分我之前没有接触过, 比较不懂, 所以就直接让codex帮我安装环境, 然后不是要执行每一步的指令嘛, 我先执行, 然后再去问codex每一步的操作是干嘛的, 把这个运行的操作搞懂。

2. 我认为读论文这部分最困难, 刚开始读的时候会碰到很多专有名词, 就要一个一个问ai, 一篇论文可能要问十几个名词, 以及硬件部分, 像P4交换机(看今年sigcomm的turbo论文时用到的), SSD (moon cake), GPU, CPU, 路由器, DRAM之类的, 我就要一个一个去b站上面找视频, 看讲解过程, 主要是看每个硬件的内部构造, 作用等, 还有一些算法的实现, 刚开始也很难理解, 比如flashattention是如何实现分块attention计算的, 像这些我感觉都很难, 但是研究明白后就是感觉很爽, 很有成就感, 其次能看到许多前人的创新的想法, 我就也很想创新, 贡献自己的idea。这也是push我的一个动力。

3. 我认为像0到1的研究, 就是看该方面的影响因素, 以KVcache为例, 他受到计算, 传输, 存储, 算法, 硬件等各个方面的影响, 如果我以后要科研, 我先看其中一个影响因素面临的比较大的问题是什么, 看看其他论文提出了哪些创新的方案, 以稀疏注意力举例, h2o提出的是把得分低的token删掉, 这就造成数据的丢失, ds最新提出的CSA2架构也是用稀疏, 但是不删token, 我认为就是一个很好的优化方法。其次, 也可以把多个因素的解决方法糅合到一起, turbo论文就很好地实现这一点。